



UNIVERSIDAD AUTÓNOMA DE SAN LUIS POTOSÍ

Facultad de Estudios Profesionales Zona Media

PRÁCTICA 13

Control de velocidad y giro de motor usando interfaz gráfica

PWM, puente H con transistores y control en tiempo real

Alumno: Cesar Gael Mancilla Estrada
Clave: 382031
Carrera: Ingeniería en Mecatrónica
Materia: Microcontroladores
Profesor: Ing. Jesús Padrón
Fecha: Abril de 2025

.^{El} PWM permite un control preciso de velocidad en motores DC"

Índice

1. Objetivo	2
2. Introducción	2
2.1. Contexto	2
2.2. Importancia	2
3. Desarrollo	3
3.1. Diagrama en Protoboard	3
3.2. Conexiones del sistema	4
3.3. CMakeLists.txt	4
4. Código C para la Pico W	4
5. Código Python - Interfaz Gráfica	9
6. Preguntas de Repaso	15
7. Conclusiones	16
7.1. Aprendizajes de la práctica	16
7.2. Posibles mejoras	16
8. Referencias	16

1. Objetivo

Aplicar el PWM como método de control en motores de corriente directa para regular su velocidad, y diseñar una interfaz gráfica que se comuniquen con el microcontrolador para enviar comandos que le indiquen al motor qué velocidad y dirección de giro debe tener.

2. Introducción

2.1. Contexto

El control de velocidad y dirección de motores DC es fundamental en robótica, automatización y sistemas mecatrónicos. El PWM (Pulse Width Modulation) permite variar la potencia media entregada al motor sin pérdidas significativas. Para invertir el giro se utiliza un puente H, que en esta práctica se implementa con transistores 2N2222A.

2.2. Importancia

Esta práctica integra conceptos de electrónica de potencia (puente H), comunicación serial, diseño de interfaces gráficas con Python y programación de microcontroladores. Es un sistema completo que simula aplicaciones industriales reales.

3. Desarrollo

3.1. Diagrama en Protoboard

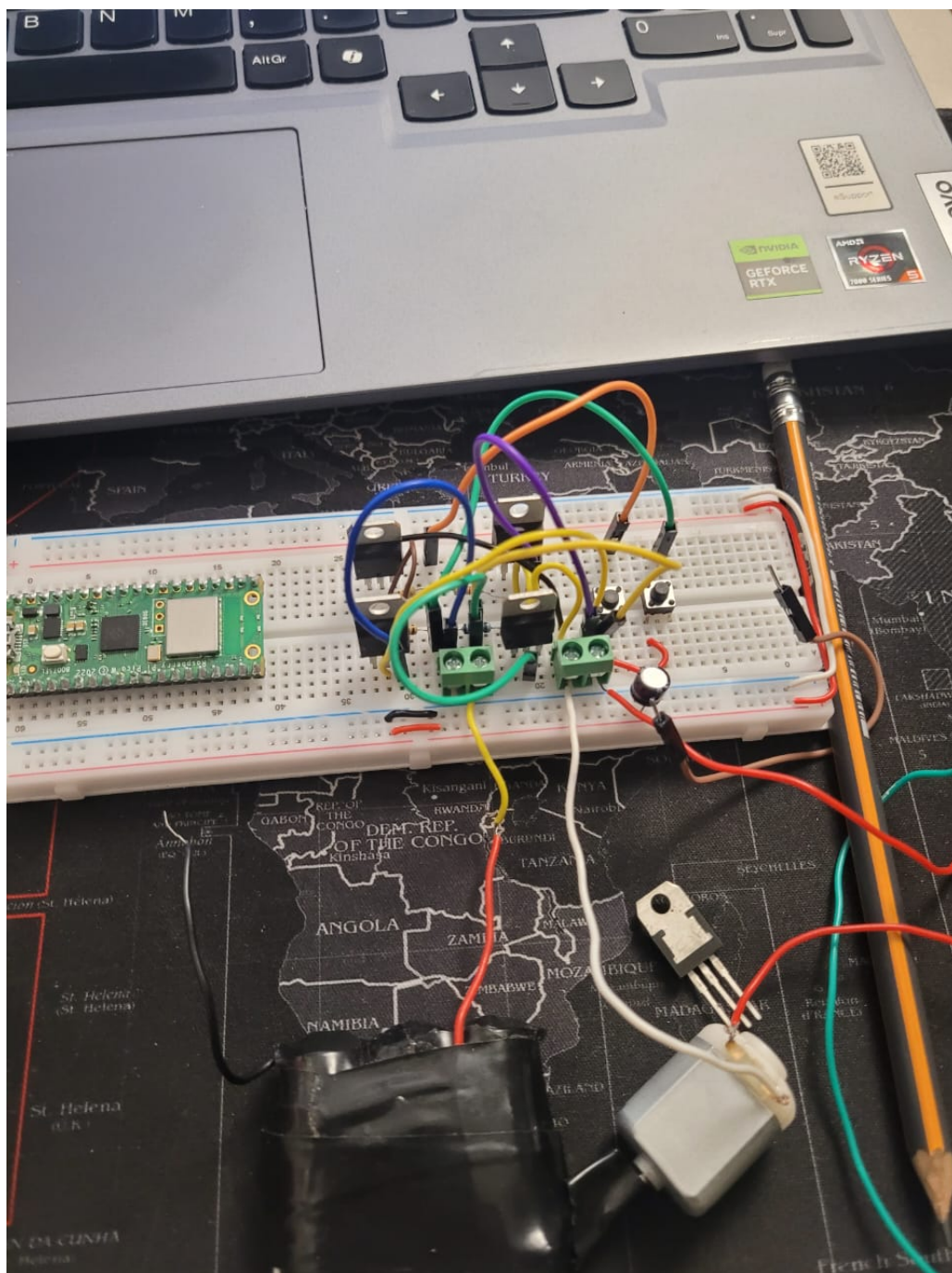


Figura 1: Conexión del puente H con transistores 2N2222A y motor DC

3.2. Conexiones del sistema

Tabla 1: Conexiones del puente H y motor

Componente	Pin Pico	Función
PWM Adelante	GP2	Control velocidad adelante
PWM Atrás	GP3	Control velocidad atrás
Enable Adelante	GP4	Habilitar giro adelante
Enable Atrás	GP5	Habilitar giro atrás
LED interno	GP25	Indicador de actividad
Motor DC	Salida puente H	Carga controlada
Alimentación motor	5V-12V externa	Fuente de potencia

3.3. CMakeLists.txt

Listing 1: Archivo CMakeLists.txt

```

1 cmake_minimum_required(VERSION 3.13)
2
3 include(pico_sdk_import.cmake)
4
5 project(motor_pwm C CXX ASM)
6
7 set(CMAKE_C_STANDARD 11)
8 set(CMAKE_CXX_STANDARD 17)
9
10 pico_sdk_init()
11
12 add_executable(motor_pwm
13     motor_pwm.c
14 )
15
16 pico_enable_stdio_usb(motor_pwm 1)
17 pico_enable_stdio_uart(motor_pwm 0)
18
19 target_link_libraries(motor_pwm
20     pico_stdlib
21     hardware_pwm
22     hardware_gpio
23 )
24
25 pico_add_extra_outputs(motor_pwm)
26
27 target_compile_options(motor_pwm PRIVATE -O2)

```

4. Código C para la Pico W

Listing 2: motor_pwm.c - Control de motor DC con puente H

```
1 /**
2  * motor_pwm.c - Control de motor DC con puente H
3  * Practica 13 - Raspberry Pi Pico W
4  */
5
6 #include <stdio.h>
7 #include <stdlib.h>
8 #include <string.h>
9 #include "pico/stdlib.h"
10 #include "hardware/pwm.h"
11 #include "hardware/gpio.h"
12
13 // Pines de control
14 #define PIN_PWM_FWD    2
15 #define PIN_PWM_REV    3
16 #define PIN_EN_FWD     4
17 #define PIN_EN_REV     5
18 #define LED_PIN        25
19
20 // Constantes
21 #define PWM_MAX        255
22 #define PWM_MIN        0
23 #define SPEED_MIN      15
24 #define BRAKE_TIME_MS  50
25
26 static int velocidad_actual = 0;
27 static bool direccion_adelante = true;
28 static bool motor_encendido = false;
29
30 void setup_pines(void) {
31     gpio_init(PIN_EN_FWD);
32     gpio_init(PIN_EN_REV);
33     gpio_set_dir(PIN_EN_FWD, GPIO_OUT);
34     gpio_set_dir(PIN_EN_REV, GPIO_OUT);
35     gpio_put(PIN_EN_FWD, 0);
36     gpio_put(PIN_EN_REV, 0);
37
38     gpio_set_function(PIN_PWM_FWD, GPIO_FUNC_PWM);
39     gpio_set_function(PIN_PWM_REV, GPIO_FUNC_PWM);
40
41     uint slice_fwd = pwm_gpio_to_slice_num(PIN_PWM_FWD);
42     uint slice_rev = pwm_gpio_to_slice_num(PIN_PWM_REV);
43
44     pwm_set_wrap(slice_fwd, PWM_MAX);
45     pwm_set_wrap(slice_rev, PWM_MAX);
46     pwm_set_clkdiv(slice_fwd, 4.0f);
47     pwm_set_clkdiv(slice_rev, 4.0f);
48     pwm_set_enabled(slice_fwd, true);
49     pwm_set_enabled(slice_rev, true);
50 }
```

```
51     pwm_set_gpio_level(PIN_PWM_FWD, 0);
52     pwm_set_gpio_level(PIN_PWM_REV, 0);
53
54     gpio_init(LED_PIN);
55     gpio_set_dir(LED_PIN, GPIO_OUT);
56     gpio_put(LED_PIN, 0);
57 }
58
59 uint16_t porcentaje_a_nivel(int velocidad) {
60     if (velocidad <= 0) return 0;
61     if (velocidad >= 100) return PWM_MAX;
62     return (velocidad * PWM_MAX) / 100;
63 }
64
65 void aplicar_velocidad(void) {
66     uint16_t nivel = porcentaje_a_nivel(velocidad_actual);
67
68     if (direccion_adelante) {
69         pwm_set_gpio_level(PIN_PWM_FWD, nivel);
70         pwm_set_gpio_level(PIN_PWM_REV, 0);
71         gpio_put(PIN_EN_FWD, nivel > 0 ? 1 : 0);
72         gpio_put(PIN_EN_REV, 0);
73     } else {
74         pwm_set_gpio_level(PIN_PWM_FWD, 0);
75         pwm_set_gpio_level(PIN_PWM_REV, nivel);
76         gpio_put(PIN_EN_FWD, 0);
77         gpio_put(PIN_EN_REV, nivel > 0 ? 1 : 0);
78     }
79
80     motor_encendido = (nivel > 0);
81 }
82
83 void frenar_motor(void) {
84     gpio_put(PIN_EN_FWD, 1);
85     gpio_put(PIN_EN_REV, 1);
86     pwm_set_gpio_level(PIN_PWM_FWD, 0);
87     pwm_set_gpio_level(PIN_PWM_REV, 0);
88     sleep_ms(BRAKE_TIME_MS);
89     gpio_put(PIN_EN_FWD, 0);
90     gpio_put(PIN_EN_REV, 0);
91 }
92
93 void motor_adelante(int velocidad) {
94     if (velocidad < 0) velocidad = 0;
95     if (velocidad > 100) velocidad = 100;
96
97     if (!direccion_adelante && motor_encendido && velocidad > 0) {
98         frenar_motor();
99     }
100
101     velocidad_actual = velocidad;
```

```
102     direccion_adelante = true;
103
104     if (velocidad_actual > 0 && velocidad_actual < SPEED_MIN) {
105         uint16_t empuje = porcentaje_a_nivel(SPEED_MIN);
106         pwm_set_gpio_level(PIN_PWM_FWD, empuje);
107         gpio_put(PIN_EN_FWD, 1);
108         sleep_ms(80);
109     }
110
111     aplicar_velocidad();
112
113     gpio_put(LED_PIN, 1);
114     sleep_ms(30);
115     gpio_put(LED_PIN, 0);
116
117     printf("ACK:FWD:%d\n", velocidad_actual);
118 }
119
120 void motor_atras(int velocidad) {
121     if (velocidad < 0) velocidad = 0;
122     if (velocidad > 100) velocidad = 100;
123
124     if (direccion_adelante && motor_encendido && velocidad > 0) {
125         frenar_motor();
126     }
127
128     velocidad_actual = velocidad;
129     direccion_adelante = false;
130
131     if (velocidad_actual > 0 && velocidad_actual < SPEED_MIN) {
132         uint16_t empuje = porcentaje_a_nivel(SPEED_MIN);
133         pwm_set_gpio_level(PIN_PWM_REV, empuje);
134         gpio_put(PIN_EN_REV, 1);
135         sleep_ms(80);
136     }
137
138     aplicar_velocidad();
139
140     gpio_put(LED_PIN, 1);
141     sleep_ms(30);
142     gpio_put(LED_PIN, 0);
143
144     printf("ACK:BWD:%d\n", velocidad_actual);
145 }
146
147 void motor_stop(void) {
148     frenar_motor();
149     velocidad_actual = 0;
150     motor_encendido = false;
151
152     gpio_put(LED_PIN, 1);
```



```
153     sleep_ms(80);
154     gpio_put(LED_PIN, 0);
155     sleep_ms(80);
156     gpio_put(LED_PIN, 1);
157     sleep_ms(80);
158     gpio_put(LED_PIN, 0);
159
160     printf("ACK:STOP\n");
161 }
162
163 void procesar_comando(char *cmd) {
164     char comando = cmd[0];
165
166     switch(comando) {
167         case 'F': case 'f':
168             motor_adelante(atoi(&cmd[1]));
169             break;
170         case 'B': case 'b':
171             motor_atras(atoi(&cmd[1]));
172             break;
173         case 'S': case 's':
174             motor_stop();
175             break;
176         default:
177             printf("ERR:CMD_UNKNOWN\n");
178             break;
179     }
180 }
181
182 int main() {
183     stdio_init_all();
184     sleep_ms(3000);
185
186     setup_pines();
187     motor_stop();
188
189     printf("\n");
190     printf("=====\n");
191     printf("    PRACTICA 13 - CONTROL DE MOTOR DC\n");
192     printf("=====\n");
193     printf("Comandos:\n");
194     printf("  F<0-100>  -> Adelante\n");
195     printf("  B<0-100>  -> Atras\n");
196     printf("  S          -> Stop\n");
197     printf("=====\n");
198     printf("LISTO!\n\n");
199
200     char buffer[32];
201     int indice = 0;
202
203     while (1) {
```

```

204     int c = getchar_timeout_us(10000);
205
206     if (c != PICO_ERROR_TIMEOUT) {
207         if (c == '\n' || c == '\r') {
208             if (indice > 0) {
209                 buffer[indice] = '\0';
210                 procesar_comando(buffer);
211                 indice = 0;
212             }
213             } else if (indice < sizeof(buffer) - 1) {
214                 buffer[indice++] = (char)c;
215             }
216         }
217
218         sleep_ms(5);
219     }
220
221     return 0;
222 }

```

5. Código Python - Interfaz Gráfica

Listing 3: interfaz_motor.py - GUI para control de motor

```

1  """
2  interfaz_motor.py
3  Practica 13 - Control de motor DC con interfaz grafica
4  """
5
6  import tkinter as tk
7  from tkinter import ttk, messagebox
8  import serial
9  import serial.tools.list_ports
10 import threading
11 import time
12
13 COLORS = {
14     'bg': '#1a1a2e',
15     'card': '#16213e',
16     'accent': '#e94560',
17     'fwd': '#00ff88',
18     'rev': '#ff4444',
19     'stop': '#ffaa00',
20     'text': '#ffffff',
21     'text_sec': '#94a3b8'
22 }
23
24 class MotorApp(tk.Tk):
25
26     def __init__(self):

```

```
27     super().__init__()
28     self.title("Practica 13 - Control de Motor DC")
29     self.geometry("450x600")
30     self.resizable(False, False)
31     self.configure(bg=COLORS['bg'])
32
33     self.serial = None
34     self.connected = False
35     self.direction = "F"
36     self.current_speed = 0
37
38     self._build_ui()
39     self._refresh_ports()
40
41 def _build_ui(self):
42     # Titulo
43     tk.Label(self, text="CONTROL DE MOTOR DC",
44             font=("Arial", 18, "bold"),
45             bg=COLORS['bg'], fg=COLORS['accent']).pack(pady=15)
46
47     tk.Label(self, text="Raspberry Pi Pico W | Puente H 2N2222A",
48             font=("Arial", 9),
49             bg=COLORS['bg'], fg=COLORS['text_sec']).pack()
50
51     # Frame conexion
52     conn_frame = tk.Frame(self, bg=COLORS['card'])
53     conn_frame.pack(fill="x", padx=15, pady=10)
54
55     tk.Label(conn_frame, text="Puerto:", bg=COLORS['card'],
56             fg=COLORS['text']).pack(side="left", padx=5)
57     self.port_combo = ttk.Combobox(conn_frame, width=12, state="readonly")
58     self.port_combo.pack(side="left", padx=5)
59
60     tk.Label(conn_frame, text="Baud:", bg=COLORS['card'],
61             fg=COLORS['text']).pack(side="left", padx=5)
62     self.baud_combo = ttk.Combobox(conn_frame, width=8, state="readonly",
63                                   values=["115200"])
64     self.baud_combo.set("115200")
65     self.baud_combo.pack(side="left", padx=5)
66
67     self.btn_connect = tk.Button(conn_frame, text="Conectar",
68                                 width=10,
69                                 bg=COLORS['fwd'], fg="black",
70                                 font=("Arial", 9, "bold"),
71                                 command=self._toggle_connect)
72     self.btn_connect.pack(side="left", padx=5)
```

```

73     self.btn_refresh = tk.Button(conn_frame, text="      ",
74                                   width=3,
75                                   bg=COLORS['accent'], fg=COLORS
76                                   ['text'],
77                                   command=self._refresh_ports)
78     self.btn_refresh.pack(side="left", padx=5)
79
80     self.lbl_status = tk.Label(self, text="      Desconectado",
81                                bg=COLORS['bg'], fg=COLORS['
82                                text_sec'])
83     self.lbl_status.pack(pady=5)
84
85     tk.Frame(self, height=2, bg=COLORS['accent']).pack(fill="x
86     ", padx=15, pady=10)
87
88     # Velocidad
89     tk.Label(self, text="VELOCIDAD", font=("Arial", 12, "bold")
90             ,
91             bg=COLORS['bg'], fg=COLORS['text']).pack()
92
93     self.speed_var = tk.IntVar(value=0)
94     self.speed_slider = tk.Scale(self, from_=0, to=100, orient=
95     tk.HORIZONTAL,
96     length=300, variable=self.
97     speed_var,
98     bg=COLORS['card'], fg=COLORS['
99     text'],
100    highlightthickness=0,
101    command=self._on_speed_change)
102     self.speed_slider.pack(pady=10)
103
104     self.lbl_speed = tk.Label(self, text="0%", font=("Arial",
105     24, "bold"),
106     bg=COLORS['bg'], fg=COLORS['
107     accent'])
108     self.lbl_speed.pack()
109
110     # Botones presets
111     preset_frame = tk.Frame(self, bg=COLORS['bg'])
112     preset_frame.pack(pady=10)
113     for pct in [0, 15, 30, 50, 75, 100]:
114         btn = tk.Button(preset_frame, text=f"{pct}%", width=4,
115                         bg=COLORS['card'], fg=COLORS['text'],
116                         command=lambda v=pct: self._set_preset(v)
117                         )
118         btn.pack(side="left", padx=2)
119
120     # Direccion
121     tk.Label(self, text="DIRECCION", font=("Arial", 12, "bold")

```

```

111         bg=COLORS['bg'], fg=COLORS['text']).pack(pady=(15,
112             5))
113
114     self.lbl_direction = tk.Label(self, text="        ADELANTE",
115         font=("Arial", 16, "bold"),
116         bg=COLORS['bg'], fg=COLORS['
117             fwd'])
118
119     self.lbl_direction.pack()
120
121     self.btn_invert = tk.Button(self, text="        INVERTIR GIRO
122         ", width=20,
123         bg=COLORS['accent'], fg=COLORS['
124             text'],
125         font=("Arial", 10, "bold"),
126         command=self._invert_direction)
127
128     self.btn_invert.pack(pady=10)
129
130     # Stop
131     self.btn_stop = tk.Button(self, text="        STOP        ",
132         width=20,
133         bg=COLORS['stop'], fg="black",
134         font=("Arial", 12, "bold"),
135         command=self._stop_motor)
136
137     self.btn_stop.pack(pady=10)
138
139     # Log
140     log_frame = tk.LabelFrame(self, text=" Comunicacion ",
141         bg=COLORS['card'], fg=COLORS['
142             text'])
143
144     log_frame.pack(fill="both", expand=True, padx=15, pady=10)
145
146     self.log_text = tk.Text(log_frame, height=6,
147         bg=COLORS['bg'], fg=COLORS['text'],
148         font=("Consolas", 8),
149         wrap="word", state="disabled")
150
151     self.log_text.pack(fill="both", expand=True, padx=5, pady
152         =5)
153
154     tk.Button(log_frame, text="Limpiar", width=8,
155         bg=COLORS['accent'], fg=COLORS['text'],
156         command=self._clear_log).pack(pady=5)
157
158     def _log(self, msg):
159         timestamp = time.strftime("%H:%M:%S")
160         self.log_text.configure(state="normal")
161         self.log_text.insert("end", f"[{timestamp}] {msg}\n")
162         self.log_text.see("end")
163         self.log_text.configure(state="disabled")
164
165     def _clear_log(self):
166         self.log_text.configure(state="normal")

```

```
155         self.log_text.delete("1.0", "end")
156         self.log_text.configure(state="disabled")
157
158     def _refresh_ports(self):
159         ports = [p.device for p in serial.tools.list_ports.comports
160                 ()]
161         self.port_combo['values'] = ports
162         if ports:
163             self.port_combo.set(ports[0])
164
165     def _toggle_connect(self):
166         if self.connected:
167             self._disconnect()
168         else:
169             self._connect()
170
171     def _connect(self):
172         port = self.port_combo.get()
173         if not port:
174             messagebox.showerror("Error", "Selecciona un puerto")
175             return
176
177         try:
178             self.serial = serial.Serial(port, 115200, timeout=0.1)
179             time.sleep(2)
180             self.connected = True
181             self.btn_connect.config(text="Desconectar", bg=COLORS['
182                                     rev'])
183             self.lbl_status.config(text=f"          Conectado a {port
184                                     }", fg=COLORS['fwd'])
185             self._log(f"Conectado a {port}")
186
187             self._send_command()
188             self.reader_thread = threading.Thread(target=self.
189                 _reader, daemon=True)
190             self.reader_thread.start()
191
192         except Exception as e:
193             messagebox.showerror("Error", str(e))
194
195     def _reader(self):
196         while self.connected and self.serial:
197             try:
198                 if self.serial.in_waiting:
199                     line = self.serial.readline().decode().strip()
200                     if line:
201                         self.after(0, self._log, f"          {line}")
202             except:
203                 break
204             time.sleep(0.01)
```

```

202     def _disconnect(self):
203         self.connected = False
204         if self.serial:
205             self.serial.close()
206         self.btn_connect.config(text="Conectar", bg=COLORS['fwd'])
207         self.lbl_status.config(text="Desconectado", fg=COLORS['text_sec'])
208         self._log("Desconectado")
209
210     def _send_command(self):
211         if not self.connected or not self.serial:
212             return
213         cmd = f"{self.direction}{self.current_speed}\n"
214         try:
215             self.serial.write(cmd.encode())
216             self._log(f"{cmd.strip()}")
217         except:
218             self._disconnect()
219
220     def _on_speed_change(self, value):
221         self.current_speed = int(value)
222         self.lbl_speed.config(text=f"{self.current_speed}%")
223         self._send_command()
224
225     def _set_preset(self, value):
226         self.speed_slider.set(value)
227         self.current_speed = value
228         self.lbl_speed.config(text=f"{value}%")
229         self._send_command()
230
231     def _invert_direction(self):
232         if self.direction == "F":
233             self.direction = "B"
234             self.lbl_direction.config(text="ATRÁS", fg=COLORS['rev'])
235         else:
236             self.direction = "F"
237             self.lbl_direction.config(text="ADELANTE", fg=COLORS['fwd'])
238         self._send_command()
239         self._log(f"Dirección: {'ADELANTE' if self.direction == 'F' else 'ATRÁS'}")
240
241     def _stop_motor(self):
242         if not self.connected:
243             return
244         try:
245             self.serial.write(b"S\n")
246             self._log("STOP")
247             self.speed_slider.set(0)
248             self.current_speed = 0

```

```
249         self.lbl_speed.config(text="0%")
250     except:
251         pass
252
253 if __name__ == "__main__":
254     app = MotorApp()
255     app.mainloop()
```

6. Preguntas de Repaso

1. ¿Qué es el ciclo de trabajo en un PWM?

El ciclo de trabajo (duty cycle) es el porcentaje de tiempo que una señal PWM permanece en nivel alto durante un período. Se calcula como: $\text{Ciclo} = \frac{t_{on}}{T} \times 100\%$.

2. ¿Qué significa que el ciclo de trabajo del PWM sea de 40 %?

Significa que durante el 40 % del período la señal está en alto (3.3V) y el 60 % en bajo (0V). El motor recibirá el 40 % de la potencia máxima.

3. ¿Cómo se genera una señal PWM en un microcontrolador?

Se utiliza un temporizador que cuenta hasta un valor máximo (wrap). Un registro de comparación determina cuándo cambiar el estado de la salida. Cuando el contador es menor que el valor de comparación, la salida está en alto.

4. ¿Por qué razón no puede conectarse directamente el PWM del microcontrolador al motor?

El microcontrolador solo puede entregar una corriente máxima de aproximadamente 12 mA por pin, insuficiente para alimentar un motor DC que consume cientos de mA. Además, el motor genera picos de voltaje inverso que podrían dañar el microcontrolador.

5. ¿Cuánta corriente puede entregar el pin PWM del microcontrolador?

Los pines GPIO de la Raspberry Pi Pico W pueden entregar hasta 12 mA de forma segura, con un máximo absoluto de 50 mA. Es insuficiente para motores, por lo que se requiere un puente H o driver externo.

6. ¿Qué función tiene el puente H en el proyecto?

El puente H permite:

- Controlar la dirección de giro del motor (adelante/atrás)
- Aislar el microcontrolador de las altas corrientes del motor
- Proteger el microcontrolador de picos inversos del motor
- Permitir el frenado regenerativo del motor

7. Conclusiones

7.1. Aprendizajes de la práctica

- Se comprendió la aplicación del PWM para control de velocidad en motores DC
- Se implementó un puente H con transistores 2N2222A para control de dirección
- Se desarrolló una interfaz gráfica en Python con tkinter para control remoto
- Se implementó comunicación serial bidireccional entre PC y Pico W
- Se añadieron mejoras como frenado antes de invertir y arranque suave

7.2. Posibles mejoras

- Implementar control PID para velocidad constante independiente de carga
- Agregar sensor de corriente para protección contra sobrecarga
- Incluir encoder para control de posición
- Añadir velocímetro virtual basado en PWM
- Control mediante interfaz web desde cualquier dispositivo

8. Referencias

1. Raspberry Pi Foundation. (2024). *Raspberry Pi Pico W Datasheet*.
2. Raspberry Pi Foundation. (2024). *Pico SDK Documentation - PWM*.
3. Texas Instruments. (2024). *2N2222A Datasheet*.